SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

Duration: 1 Hour

QUESTIONS: 3 **Total Marks:30**

Annexure A

Dry Run Evaluation

Code of Conduct:

I A.R. Afreen Fariha / 192571025 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

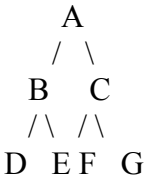
Signature of the student:

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph:



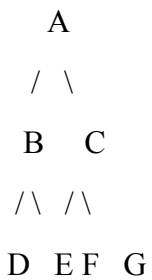
Task: Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

Iteration	Queue	Visited Node
1		
2		
3		
4		
5		
6		
7		

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

2. Depth-First Search (DFS) Using the same graph,



Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform **Depth-First Search (DFS)** starting from **A**.

Complete the table.

3. **Initial State :**
   1 3 6
   5 0 2
   4 7 8

Goal State :

1 2 3
4 5 6
7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.
Fill in the table.

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1			
2			
3			
4			
5			

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

ANSWER BOOKLET – CSA17 Artificial Intelligence

Name: A.R. Afreen Fariha **Reg No:** 192571025

Assessment Tool 3 – Annexure A: Dry Run Evaluation (CO2)

Course outcome covered: CO2 – search and game-solving techniques (BFS, DFS and state-space search with queue / stack data structures).

Q.	Problem	Algorithm / data structure
1	Search on the 7-node tree A–G	Breadth-First Search with a FIFO queue
2	Search on the same tree	Depth-First Search with a LIFO stack
3	8-puzzle: initial state 136/502/478 to goal 123/456/780	Breadth-First Search on the state space (queue before/after expansion)

Conventions used in all dry runs

- **Tree:** A is the root; B and C are its children; B has children D and E; C has children F and G. Children are always taken **left to right**.
- **Queue (BFS):** first in, first out. The front of the queue is written on the left.
- **Stack (DFS):** last in, first out. The top of the stack is written on the left. Children are pushed right to left so that the left child is popped first.
- **Visited node:** the node removed from the queue/stack and expanded in that iteration.
- **8-puzzle:** the blank is written as 0. Moves are the blank moving **Up, Down, Left, Right**, tried in this order. A state already generated earlier (including the parent) is not added again. The goal test is applied when a state is removed from the queue.

Question 1: Breadth-First Search (BFS)

1.0 Idea of the algorithm

Breadth-First Search explores the graph **level by level**. It keeps a **FIFO queue**: the node at the front is removed and visited, and its unvisited children are added at the back. All nodes at depth d are therefore visited before any node at depth d + 1.

```
BFS(start):
    queue <- [start]
    while queue is not empty:
        node <- dequeue(queue)            # front of the queue
        visit(node)
        for each child of node (left to right):
            enqueue(child)                # back of the queue
```

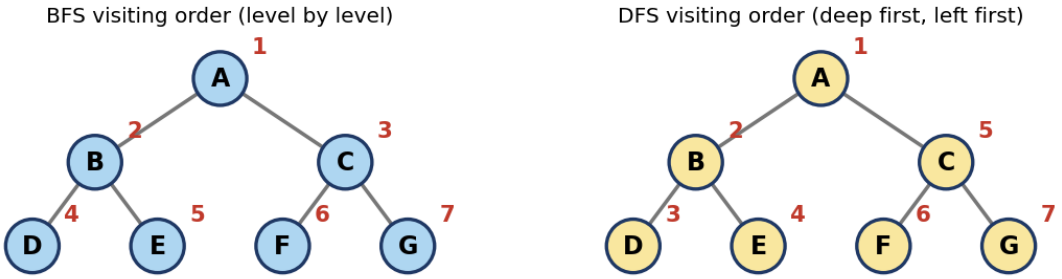


Figure 1: Visiting order of BFS (left) and DFS (right) on the given tree; red numbers show the iteration

1.1 Dry run table

Iteration	Queue (before removing front)	Visited node	Queue after adding children
1	A	A	B, C
2	B, C	B	C, D, E
3	C, D, E	C	D, E, F, G
4	D, E, F, G	D	E, F, G
5	E, F, G	E	F, G
6	F, G	F	G
7	G	G	(empty)

The table shows the queue *before* the front is removed (so the first column of the original table), the node visited, and the queue after the children have been added.

1.2 Answers to the questions

Question 1: BFS traversal order

BFS traversal order
A → B → C → D → E → F → G

Level 0: A. Level 1: B, C. Level 2: D, E, F, G. Nodes are visited in this level order, and within a level from left to right.

Question 2: Queue contents after each iteration

Iteration	Node removed and visited	Children added	Queue after the iteration
1	A	B, C	[B, C]
2	B	D, E	[C, D, E]
3	C	F, G	[D, E, F, G]
4	D	– (leaf)	[E, F, G]
5	E	– (leaf)	[F, G]
6	F	– (leaf)	[G]
7	G	– (leaf)	[ ]

Properties of BFS (supporting discussion)

- **Complete:** it finds a solution if one exists, because every level is eventually explored (finite branching factor).
- **Optimal for unit step cost:** the first time the goal is reached it is at the smallest depth.
- **Time and space:** $O(b^d)$ both; here b = 2 and the whole tree has 7 nodes. The queue never held more than 4 nodes (iteration 4), which is the width of the widest level.

Question 2: Depth-First Search (DFS)

2.0 Idea of the algorithm

Depth-First Search always goes **as deep as possible** along one branch before backing up. It uses a **LIFO stack**: the node on the top is removed and visited, and its children are pushed on the stack. The children are pushed from right to left so that the *left* child ends up on top and is explored first.

```
DFS(start):
    stack <- [start]
    while stack is not empty:
        node <- pop(stack)                # top of the stack
        visit(node)
        for each child of node (right to left):
            push(child)                   # so the left child is popped first
```

2.1 Dry run table

Iteration	Stack (top on the left, before popping)	Visited node	Stack after pushing children
1	A	A	B, C
2	B, C	B	D, E, C
3	D, E, C	D	E, C
4	E, C	E	C
5	C	C	F, G
6	F, G	F	G
7	G	G	(empty)

2.2 Result

DFS traversal order
A → B → D → E → C → F → G

Explanation. After A, the stack holds B on top, so DFS goes down the left branch: B, then D (a leaf). It backtracks to E (the next element on the stack), which is also a leaf, and only then does it return to C and explore C's subtree: F and then G. The stack never held more than 3 nodes (iteration 2), the height of the tree plus the pending siblings, which is why DFS needs much less memory than BFS.

2.3 Comparison of BFS and DFS on this graph

Feature	BFS	DFS
Data structure	FIFO queue	LIFO stack (or recursion)
Visiting order	A B C D E F G	A B D E C F G
Maximum frontier size	4 nodes (queue)	3 nodes (stack)
Space complexity	$O(b^d)$	O(b·m), m = maximum depth
Finds shallowest goal?	Yes	Not guaranteed

Feature	BFS	DFS
Complete?	Yes (finite b)	Only in finite / acyclic spaces or with a visited list
Best used when	Shortest path in an unweighted graph is needed	Memory is limited, or any solution is acceptable

If the goal were node E, BFS would reach it in iteration 5 and DFS in iteration 4; if the goal were C, BFS needs iteration 3 but DFS needs iteration 5. This shows that neither algorithm is faster in every case; BFS is the one with the optimality guarantee.

Question 3: 8-Puzzle with Breadth-First Search

3.0 Problem formulation

Element	Definition
State	A 3 × 3 arrangement of the tiles 1–8 and the blank (0)
Initial state	1 3 6 / 5 0 2 / 4 7 8
Goal state	1 2 3 / 4 5 6 / 7 8 0
Actions	Slide the blank Up, Down, Left or Right (when it stays inside the board); tried in this order
Step cost	1 for every move (unweighted)
Solvability check	Writing the initial tiles in a row without the blank gives 1 3 6 5 2 4 7 8, which has 6 inversions (even), as does the goal (0 inversions), so the goal is reachable.

Notation. States are named S0, S1, S2, … in the order in which they are generated. A state is written as three rows separated by "/", for example S0 = 136/502/478.

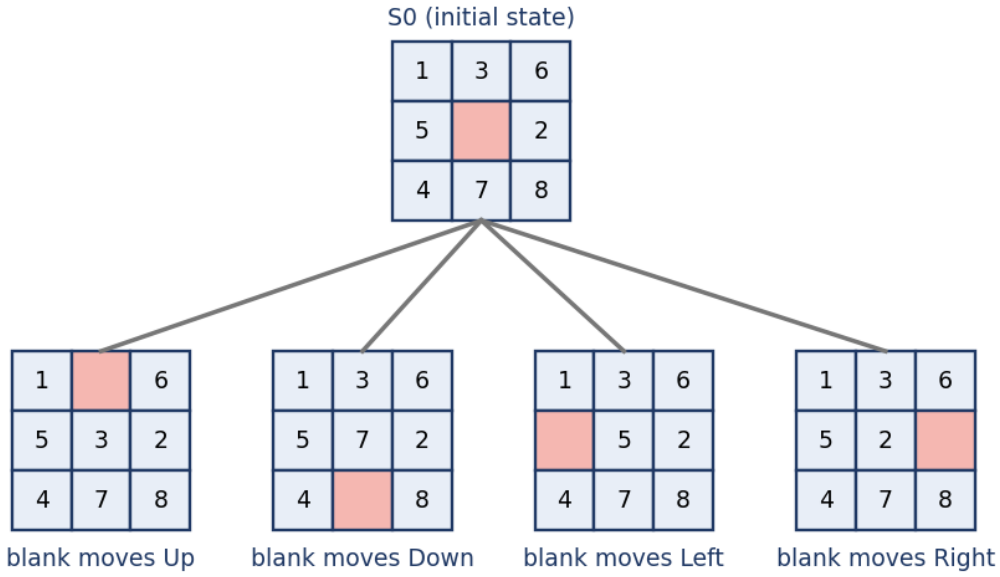


Figure 2: Initial state S0 and its four children (blank moved Up, Down, Left, Right)

3.1 Dry run table (first five iterations)

Iter.	Current state (expanded)	Queue before expansion	Children generated	Queue after expansion
1	S0 (136/502/478)	S0	S1 (Up), S2 (Down), S3 (Left), S4 (Right)	S1, S2, S3, S4
2	S1 (106/532/478)	S1, S2, S3, S4	S5 (Left), S6 (Right)	S2, S3, S4, S5, S6
3	S2 (136/572/408)	S2, S3, S4, S5, S6	S7 (Left), S8 (Right)	S3, S4, S5, S6, S7, S8
4	S3 (136/052/478)	S3, S4, S5, S6, S7, S8	S9 (Up), S10 (Down)	S4, S5, S6, S7, S8, S9, S10
5	S4 (136/520/478)	S4, S5, S6, S7, S8, S9, S10	S11 (Up), S12 (Down)	S5, S6, S7, S8, S9, S10, S11, S12

Legend of the states appearing in the table:

State	Board (row/row/row)	Created by
S0	136/502/478	Initial state
S1	106/532/478	S0 with blank Up
S2	136/572/408	S0 with blank Down
S3	136/052/478	S0 with blank Left
S4	136/520/478	S0 with blank Right
S5	016/532/478	S1 with blank Left
S6	160/532/478	S1 with blank Right
S7	136/572/048	S2 with blank Left
S8	136/572/480	S2 with blank Right
S9	036/152/478	S3 with blank Up
S10	136/452/078	S3 with blank Down
S11	130/526/478	S4 with blank Up
S12	136/528/470	S4 with blank Down

Check of iteration 2. S1 = 106/532/478 has the blank at the top-middle position, so the Down move would recreate the parent S0 (already generated) and is skipped; only Left (S5) and Right (S6) are new. The same rule explains why S2, S3 and S4 each generate only two new children in iterations 3, 4 and 5.

After five iterations 5 states have been expanded (S0–S4), 13 states have been generated (S0–S12) and the queue holds the 8 states S5–S12, all at depth 2. **The goal has not yet been reached**, because it lies at depth 8; BFS therefore continues, and the table above is the beginning of the same trace, which ends as summarised next.

3.2 Continuing BFS until the goal is reached

Depth	States expanded at this depth	Cumulative states expanded	States generated at this depth
0	1	1	1

Depth	States expanded at this depth	Cumulative states expanded	States generated at this depth
1	4	5	4
2	8	13	8
3	8	21	8
4	16	37	16
5	32	69	32
6	60	129	60
7	72	201	72
8	110	311	136
9	0 (search stops earlier)	311	163 (children of the depth-8 states expanded before the goal)

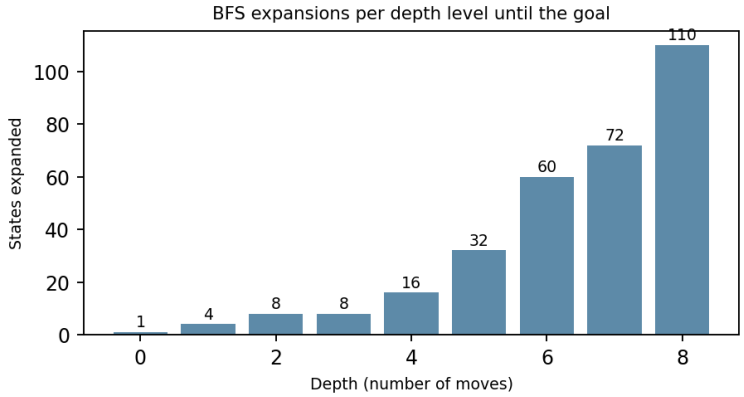


Figure 3: Number of states expanded at each depth level

All 1 + 4 + 8 + 8 + 16 + 32 + 60 + 72 = 201 states of depths 0–7 are expanded first. Depth 8 contains 136 states; they are expanded in generation order and the goal state 123/456/780 is the 110th of them, i.e. the **311th expanded state overall** (201 + 110). The remaining 26 depth-8 states and the 163 depth-9 states generated meanwhile are still in the queue (26 + 163 = 189). The goal test succeeds at iteration 311.

3.3 Answers to the questions

Question 1: Which node is expanded in each iteration?

Iterations 1 to 5 expand S0, S1, S2, S3 and S4 respectively (see the table). In general, BFS expands states in the order in which they were generated (FIFO): depth 0, then the 4 states of depth 1, then the 8 of depth 2 (iterations 6–13), and so on, level by level. The goal state is expanded in iteration 311.

Question 2: How many states are generated in total?

Total number of states generated
500 states (the initial state plus 499 children) are generated by the time the goal is removed from the queue: 311 states have been expanded and 189 remain in the queue.
If the goal test were applied when a state is generated (instead of when it is expanded), the goal appears as the 311th generated state, during the 187th expansion, so only 311 states would be generated.

Repeated states are not counted (each distinct arrangement is generated once). Without that check the number would be larger, since the blank can always move back to the previous position.

Question 3: Complete solution path from the initial state to the goal state

The shortest solution has **8 moves**: **Right, Up, Left, Down, Left, Down, Right, Right** (movement of the blank).

Step	Move of blank	Resulting state
0	–	136/502/478 (initial)
1	Right	136/520/478
2	Up	130/526/478
3	Left	103/526/478
4	Down	123/506/478
5	Left	123/056/478
6	Down	123/456/078
7	Right	123/456/708
8	Right	123/456/780 (goal)

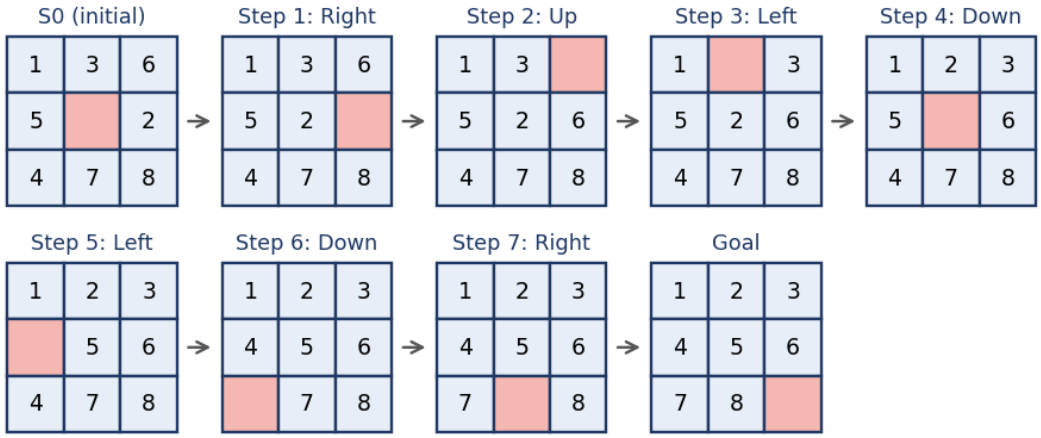


Figure 4: Solution path found by BFS (red cell = blank)

Written as boards: 136/502/478 → 136/520/478 → 130/526/478 → 103/526/478 → 123/506/478 → 123/056/478 → 123/456/078 → 123/456/708 → 123/456/780.

Question 4: Why does BFS always find the shortest solution in an unweighted 8-puzzle?

1. **Every move costs 1,** so the cost of a path equals its number of moves, i.e. the depth of the state in the search tree.
2. **BFS expands states in order of non-decreasing depth.** Because the queue is FIFO, all states at depth d are removed from the queue before any state at depth d + 1, since the children of depth-d states are always added behind the states already waiting.
3. **The first time the goal is reached is at the smallest possible depth.** Suppose a shorter solution of depth k existed while BFS returned a solution of depth m > k. The state at depth k on the shorter path would be generated and would sit in the queue ahead of every depth-(k+1) state, so it would have been found before the depth-m solution, a contradiction.
4. **Completeness:** the 8-puzzle has finitely many states (9!/2 = 181,440 reachable) and a finite branching factor (at most 4), so BFS terminates and finds a solution whenever one exists.

In this problem BFS therefore guarantees that 8 moves is the minimum for this start state. The price is memory: BFS stores the whole frontier, and the queue held 189 states when the goal was found (growth about b^d), so for harder puzzles informed methods such as A* with the Manhattan-distance heuristic are preferred, while BFS is still the correct choice when a guaranteed shortest solution and a small state space make it affordable.

3.4 Summary of results

Item	Result
Goal depth / minimum number of moves	8
States expanded until the goal is dequeued	311
States generated	500 (311 if the goal is tested on generation)
States left in the queue	189
Solution path	Right, Up, Left, Down, Left, Down, Right, Right